

Final Project Plan: Organizational Hierarchy Management API

Combined & De-Conflicted Plan — Synthesized from 8 AI Recommendations + Original Brief

How This Plan Was Built

Eight independent AI-generated plans (ChatGPT, Claude, Copilot, DeepSeek, Gemini, Google AI Studio, Grok, Perplexity) were reviewed alongside the original Project Brief. Most of them agreed on the fundamentals. Three areas had real disagreement. This document keeps the consensus, resolves the conflicts with explicit reasoning, and presents a single, interview-defensible plan — no contradictions left unresolved.

Where All Plans Agreed (No Decision Needed)

- **Stack: Option A** — Next.js + NestJS + PostgreSQL + TypeORM. All eight plans converged here independently.
- **Layered backend:** Controller → Service → Repository → Entity/DTO, with `class-validator` DTOs, a global `ValidationPipe`, and a global exception filter returning a consistent error shape.
- **Cycle prevention** on parent updates: reject the update if the new parent is a descendant of the node being moved.
- **Environment-based DB configuration**, replacing the scaffold's hardcoded credentials.
- **Controller unit tests** (a hard requirement per the brief) plus polished Swagger documentation.
- **Adjacency list** (a `parentId` column) as the underlying storage — every plan uses this as the base, even the ones that layer something on top of it.

Where the Plans Collided — and the Resolution

1. Tree Storage: Plain Adjacency List + Recursive CTE (Not TypeORM `@Tree()`)

Gemini was the outlier, recommending TypeORM's built-in `@Tree("closure-table")` for single-query tree fetches. This plan overrides that. Six of the eight plans (Claude, Copilot, DeepSeek, Google AI Studio, Grok, Perplexity) independently chose a plain `parentId` column plus a recursive CTE instead, and that reasoning wins: `@Tree()` is a black box. If an interviewer asks how a query works, the goal is to explain a `WITH RECURSIVE SQL` statement you wrote yourself, not a decorator you imported. Closure tables are also harder to keep in sync on every re-parent or delete, which works against the brief's requirement that the client can restructure the hierarchy at any time.

Decision: Position entity with a self-referencing parentId. Use a recursive CTE for subtree/descendant queries, and a single flat fetch plus an in-memory parent-map pass ($O(n)$) for “get the whole tree.” Materialized paths and closure tables are mentioned only as a future optimization if read volume ever became very large — that satisfies the underlying concern Gemini raised without taking on the complexity now.

2. Deletion Policy: Reassign Children to Parent, Block CEO Deletion, Soft Delete

This was the biggest split across the eight plans:

- Copilot recommended cascade delete (wipe the whole branch).
- ChatGPT recommended blocking deletion entirely if children exist (HTTP 409).
- Claude, DeepSeek, Gemini, Google AI Studio, Grok, and Perplexity recommended reassigning children to the deleted node’s parent.
- Gemini additionally suggested letting the *user* pick cascade vs. restrict via a frontend dropdown.
- DeepSeek recommended reassignment in general, but cascade specifically if the node being deleted is the root.

Resolution: the majority position, simplified. Cascade silently destroys data; pure-restrict fights the “restructure at any time” requirement. Gemini’s “let the user choose strategy” idea is explicitly rejected — it sounds flexible, but it means defending two deletion algorithms in the interview instead of one, for little real functional gain. DeepSeek’s CEO-cascade edge case is also rejected — cascading away the entire hierarchy because the CEO was deleted is more dangerous than simply disallowing that operation outright.

Decision:

- Deleting a node with children re-points those children to that node’s own parent (one level up), preserving the rest of the tree.
- Deleting the CEO (root, parentId === null) is blocked with a clear error — there is no parent to promote children to, and the org must always have exactly one root.
- Actual row removal uses **soft delete** (@DeleteDateColumn()), not a hard delete — auditable, trivial to add, and a clean answer to “what if someone deletes the wrong thing.”

3. Frontend Tree Rendering: Custom Recursive Component First, Library Only as a Stretch Goal

Copilot, Gemini, DeepSeek, and Google AI Studio all suggested a charting library (react-d3-tree, React Flow, react-organizational-chart). Claude and Grok suggested a plain recursive component. Given the goal of “not too complicated, not too simple,” the simpler route wins: a hand-rolled recursive <OrgTree> component (nested list with expand/collapse) is functionally identical from an interviewer’s point of view, carries zero integration risk, and every line of it can be explained on the spot. A charting library is a stretch goal only if core functionality is finished early.

Final Data Model

```
@Entity('position')
export class Position {
  @PrimaryGeneratedColumn('uuid')
  id: string;
```

```

@Column()
name: string;

@Column({ type: 'text', nullable: true })
description: string;

@Column({ nullable: true })
parentId: string | null; // null only for CEO

@ManyToOne(() => Position, p => p.children, { nullable: true })
@JoinColumn({ name: 'parentId' })
parent: Position;

@OneToMany(() => Position, p => p.parent)
children: Position[];

@Column({ nullable: true })
department: string; // optional extra – easy talking
point, low cost

@Column({ default: 0 })
level: number; // denormalized depth,
recalculated on write

@CreateDateColumn()
createdAt: Date;

@UpdateDateColumn()
updatedAt: Date;

@DeleteDateColumn()
deletedAt: Date; // soft delete
}

```

department and level were kept because they appeared across multiple independent plans and cost almost nothing to add. level in particular answers “how do you avoid recalculating depth on every read.”

Backend Folder Structure (NestJS)

```

src/
├─ main.ts # Global ValidationPipe,
Swagger setup
├─ app.module.ts # Clean imports only
├─ config/
│   └─ configuration.ts # env-based DB config
├─ common/
│   └─ filters/
│       └─ http-exception.filter.ts # consistent error shape
│   └─ interceptors/
│       └─ logging.interceptor.ts
├─ positions/
│   └─ positions.module.ts
│   └─ positions.controller.ts # HTTP + Swagger decorators
only
├─ positions.service.ts # business rules: tree build,
deletion policy, cycle check
├─ positions.repository.ts # recursive CTE queries live
here
├─ entities/
│   └─ position.entity.ts
├─ dto/
│   └─ create-position.dto.ts
│   └─ update-position.dto.ts
│   └─ position-response.dto.ts

```

└─ positions.controller.spec.ts

This is a layered structure rather than full DDD. Gemini explicitly warned against full DDD with aggregate roots tripping up a candidate mid-interview, and DeepSeek's domain/application/infrastructure/interfaces split — while impressive on paper — is a lot of ceremony to defend live for what is fundamentally a CRUD-plus-tree problem. This structure still visibly satisfies “separation of concerns” without that risk.

Final API Endpoints

Method	Path	Purpose
POST	/positions	Create
GET	/positions	Flat list, optional ?search=name
GET	/positions/tree	Full nested hierarchy
GET	/positions/:id	Single position detail
GET	/positions/:id/children	Direct children only
GET	/positions/:id/ancestors	Reporting line up to CEO (breadcrumb)
PUT	/positions/:id	Update (including reparenting — cycle-checked)
DELETE	/positions/:id	Delete with reassign-to-parent policy

A few suggestions from the individual plans were deliberately dropped from the core set: a separate PATCH /:id/move endpoint (folded into PUT, since reparenting is just an update), bulk upload, a stats endpoint, and a department-filter endpoint. These are real ideas, but not core — see the “if time allows” list below. Fewer endpoints that are fully understood beats more endpoints that are shaky under questioning.

Frontend Structure (Next.js)

```
src/
├─ app/
│   ├─ page.tsx                # tree view (default)
│   └─ positions/[id]/page.tsx # detail/edit
├─ components/
│   ├─ OrgTree.tsx             # recursive, hand-rolled tree
│   └─ PositionTable.tsx       # flat/searchable table view
(toggle with tree)
├─ PositionForm.tsx           # create/edit, react-hook-form + zod
├─ DeleteConfirmDialog.tsx    # warns "children will be reassigned to X"
├─ lib/
│   └─ api.ts                  # typed fetch client
└─ types/
    └─ position.ts
```

React Query (or SWR) is used for data fetching and caching — every plan that addressed frontend state agreed on this category of tool.

Testing Strategy

- Controller unit tests (required) with a mocked service — Jest + Nest testing module.
- Service-level tests specifically for: cycle prevention, reassign-on-delete, and “cannot delete root.” This is the logic most likely to get probed in the interview, so tests here double as interview preparation.

Explicit “If Time Allows” List

These ideas are good but deliberately deferred so they don’t consume the one-week budget:

- **Audit/history table** logging every change (from DeepSeek) — a second table and more write-path logic; add only once core CRUD, tree, and deletion are solid.
- **Redis caching** of the tree (from DeepSeek) — premature for this dataset size; keep as a future-scaling talking point instead.
- **Drag-and-drop reparenting, PDF/PNG export, charting libraries** (from Copilot, DeepSeek, Gemini, Google AI Studio) — visually nice, but functionally redundant with form-based reparenting.
- **Docker Compose setup** (from DeepSeek) — good polish, zero functional risk, a good last-day addition if time remains.
- **Department-filter and bulk-upload endpoints** (from ChatGPT, DeepSeek) — easy once the core service exists; good filler if the core is finished early.

One-Week Timeline

Day	Focus
1	Scaffold cleanup, entity, DTOs, env config, basic CRUD
2	Tree query, ancestors, children, cycle check, deletion logic
3	Controller + service tests, exception filter, Swagger polish
4	Next.js scaffold, API client, tree + table views
5	Forms (create/edit/delete), validation, search
6	Polish, edge cases, README (architecture + decisions)
7	Buffer + rehearse explaining the architecture out loud

Interview Answers (Consolidated — No Contradictions)

Why adjacency list + recursive CTE, not a tree-table library?

Simpler to keep consistent under frequent restructuring, and the query can be explained directly rather than pointing at a library feature.

Why reassign children on delete instead of cascade or block?

Cascade destroys data unnecessarily; pure-block fights the “restructure at any time” requirement; reassignment keeps the org intact and matches how real reorganizations work.

Why block CEO deletion specifically? There is no parent to promote children to, and an org chart needs exactly one root.

How do you prevent cycles? Before any parent update, check that the new parent is not already a descendant of the node being moved.

What would you add with more time? Audit log, caching, and role-based access control — named explicitly as deferred, not forgotten.